

DAY 2 · 下午第一节 · SDLC 实弹(1) · 120 MIN

Engine 设计好了——现在让它跑起来



实验任务 (统一基线)

为错题本开发「拍照识别」功能

用户拍照 → 上传图片 → Bedrock 识别 → 返回结构化 JSON

覆盖: API 设计 + 识别逻辑 + 错误处理 + 置信度判定 · 上半场跑 EVALUATE + PLAN

把 Engine 注入 Agent —— 它不会自动读 stages.md

和 Knowledge 一样：Engine 文件存在磁盘上，agent 不会自动读。需要一个明确触发入口。

```
# engine/SKILL.md
---
trigger: "开始任务" OR "run engine"
        OR 任务下发时
---
```

执行流程

1. 读 engine/STATE.md 确定当前位置
2. 新任务：确定 profile → 初始化
STATE.md → 进入第一阶段
3. 续做：读 STATE.md → 继续该阶段
4. 每阶段结束：
 - a. 产出 artifact
 - b. 跑出口 gate (读 gates.md)
 - c. 过 → 更新 STATE → 下一阶段
 - d. 失败 → 原地修复 / 回退

方式 1 · SessionStart hook

注入 STATE.md + 当前阶段的执行指引

方式 2 · Engine SKILL.md ★

作为触发入口 —— 推荐方式

为什么需要它

agent 看到了也可能觉得「我知道该怎么做」然后跳过，必须显式触发

Stage 1: EVALUATE — 在跑错方向之前停下来

操作步骤

- ① 开新 session (验证 Knowledge 注入)
- ② 告诉 agent: "用 feature profile 跑 engine, 任务是: 为错题本开发拍照识别功能"
- ③ 观察 agent 进入 EVALUATE 的行为

⚡ 判断时刻 (先自己判断, 再看 agent)

在 agent 输出 AC 之前——你自己先写 3 条 AC (30 秒)

对比: 你写的和 agent 写的有什么不同? 谁更可判定?

差异 = 教学素材: 为什么你们的判断不同?

Agent 在 EVALUATE 应做

- 阅读 PRODUCT.md 理解业务
- 澄清模糊点 (如果有)
- 输出 artifact: What / Why / AC / Risks / 不可逆决策

观察点

- 是否真读了 DDD? (还是凭空编需求)
- AC 是否可判定? ("体验好"不算)
- 是否识别了不可逆决策?

若 agent 跳过 EVALUATE 直接写代码 → 这正是 Engine 要解决的问题。提醒它「请按 Engine 流程走, 先完成 EVALUATE」。

G1 判定 —— EVALUATE 产出物合格吗？

G1 检查清单

- ✓ artifact 文件存在
- ✓ AC 数量 ≥ 3
- ✓ 每条 AC 可判定
- ✓ 不可逆决策已声明

现场可能发生

✓ 通过

大部分学员第一次可能通过（EVALUATE 相对简单）

✗ 不通过（最常见：AC 不够可判定）

纠正 → 原地修复 → 再检查；这过程本身就是 corrections 的来源

G1 的价值是什么？

没有 G1

agent 模糊地「理解了需求」就开始设计

有了 G1

必须输出可检查的 artifact，强制把理解显性化

全班一次通过 = principles 生效了；有人不通过 = 更好的教学时刻。

Stage 2: PLAN — 在不可逆决策之前停下来想清楚

Agent 在 PLAN 应做

- 读 TECH.md 了解技术约束
- 为每个 AC 设计实现方案
- 做不可逆决策 (记录 ADR)
- 设计接口 (API schema)
- 识别风险和缓解方案

关键观察点

agent 是否参考了 TECH.md 中的 ADR?

不可逆决策记录是否完整 (理由 + 代价 + 回退)? 每个 AC 是否映射到了具体实现?

Plan: 拍照识别功能

技术方案

- API: POST /api/recognize
- 输入: multipart/form-data (image)
- 输出: JSON { question_text, confidence, knowledge_points[] }

不可逆决策 (ADR)

- [ADR-003] 同步调用 Bedrock
 - 理由: MVP 简单优先, <5s 满足 AC
 - 代价: 并发高时可能超时
 - 回退: P99>5s → 重构为异步

风险

- Bedrock 冷启动首调超时
- 图片模糊/倾斜导致识别失败

每个 AC 的实现映射

- AC1 5s内返回 → 同步+超时配置
- AC3 低conf标待确认 → 后处理

G2 — 独立视角找致命缺陷

G2 是 L2 级别 —— 需要独立审查。方案合理性不能由产出者自己说了算。

你是一个严格的技术审查者。

请审查以下技术方案。

你的目标是找出：

- 1. 致命缺陷（无法交付/需重写）
- 2. 重大遗漏（AC 没被覆盖）
- 3. 风险低估（无可行缓解方案）

找不到致命缺陷 → 判定：PASS

找到致命缺陷 → FAIL + 具体问题

方法 1 · session 内调用

加一段 prompt：以 devil's advocate 角色审查刚才方案

方法 2 · 人工审查 (L3 降级)

你自己花 2 分钟看：遗漏？AC 覆盖？风险可控？

结果	动作
PASS	进入 BUILD

G2 抓住遗漏 → 方案修正后继续；G2 没抓住、拖到 BUILD/VERIFY 才暴露 → 说明 G2 太弱，下次加强。

无论如何——记录这个 G2 的效果。它是 Engine 进化的数据。

前半段完成 —— 你已经产出了什么

```
my-agentos/  
├── engine/  
│   └── STATE.md  
│       当前: BUILD, G1✅ G2✅  
├── spec/  
│   └── recognize/  
│       ├── evaluate.md  
│       │   ← EVALUATE 产出物  
│       └── plan.md  
│           ← PLAN 产出物 (含 ADR)  
├── knowledge/  
│   └── TECH.md  
│       ← 可能被更新 (新 ADR)  
└── corrections.log  
    ← 新增 N 条 corrections
```

PLAN 做了新 ADR

→ 更新 TECH.md (Knowledge 被填充)

你纠正了方案遗漏

→ corrections.log 增长 (蒸馏原料)

这就是 Flywheel

Engine 运行 → Knowledge 变丰富

不是你手动维护 Knowledge, 是 Engine 运行时自然产生的副产品。



5 MIN 快速复盘

你纠正了什么 —— 初步 pattern

典型 corrections (匿名举手收集)

- 方案遗漏类：没考虑 x 情况
- 格式类：artifact 不完整
- 引用类：没参考 DDD 文档
- 深度类：AC 写得太模糊

昨天 · 代码层面	今天 · 设计层面
❓❓❓❓❓❓❓❓❓	方案遗漏、决策没记录
不同阶段产生不同类型的 corrections → Engine 的阶段划分有意义	

现在做一件事：标记你认为的 top-3 correction patterns —— 明天上午蒸馏工坊的输入。